



Université Assane Seck de Ziguinchor
UFR Sciences et Technologies
Département Informatique

Chapitre 6 : Algorithmes de parcours de graphes
Cours de Prof. Youssou DIENG
Email : ydieng@univ-zig.sn

30. juli 2023

1 Introduction

L'objectif de cette section est de présenter quelques méthodes de parcours de graphe. Le parcours d'un graphe consiste à emprunter de façon systématique les arcs du graphe dans l'objectif de visiter les sommets du graphe. Un algorithme de parcours de graphe permet de mettre en évidence plusieurs caractéristiques de la structure du graphe. Ainsi, pour obtenir les caractéristiques structurelles des graphes, les algorithmes utilisent souvent le résultat d'un parcours. D'autres sont simplement des adaptations des algorithmes élémentaires de parcours de graphe. C'est ce qui explique le fait que les techniques de parcours de graphe soient au cœur des algorithmes de graphe.

Dans ce chapitre, nous présentons les deux algorithmes de base de parcours de graphes. Le premier algorithme présenté est **le parcours en largeur**. Une application de cet algorithme, présentée dans ce chapitre, est comment créer une arborescence de parcours en largeur. Le deuxième algorithme présenté est **le parcours en profondeur**. Nous présenterons aussi quelques résultats fondamentaux sur l'ordre de visite des sommets. Comme application de ce parcours, le prétexte qui sera utilisé est le tri topologique d'un graphe orienté sans circuit.

1.1 Définitions sur les parcours

Un **chemin** μ de longueur q est une séquence de q arcs $(u_1, u_2, \dots, u_q)$. L'extrémité initiale de u_i doit être l'extrémité terminale de u_{i-1} si $i \geq 1$, et l'extrémité terminale de u_i doit être l'extrémité initiale de u_{i+1} si $i \leq q$. Un chemin fermé est un **circuit**. Un arc peut être vu comme un chemin de longueur 1, et une boucle comme un circuit de longueur 1.

Un sommet y **descendant** de x s'il est situé sur un chemin d'origine x . On dit aussi que x est **ascendant**, ou **ancêtre** des y . Notez que, contrairement au monde vivant, un sommet peut être à la fois ascendant et descendant d'un autre, s'ils sont situés sur un même circuit.

Une **chaîne** de longueur q est une séquence de q arêtes $[e_1, e_2, \dots, e_q]$ e_i a une extrémité commune avec e_{i-1} si $i \geq 1$ et une autre avec e_{i+1} si $i \leq q$. Une chaîne fermée est un **cycle**. Une arête est une chaîne de longueur 1. On peut considérer des chaînes sur un Graphe orienté en parlant du graphe non orienté associés.

Le terme de **parcours** regroupe les chemins, les circuits, les chaînes et les cycles. Un parcours est **élémentaire** s'il emprunte qu'une fois ses sommets. Dans un graphe simple, on peut définir un parcours par une suite de sommets, en répétant le premier sommet à la fin pour un cycle ou un circuit : ainsi (x, y) désigne un chemin réduit à l'arc (x, y) , tandis que (x, y, z) indique un circuit passant sur deux arcs (x, y) et (y, z) . Par convention, un simple sommet sera considéré comme un parcours de longueur 0.

2 Parcours en largeur

L'algorithme du parcours en largeur est l'un des algorithmes de parcours les plus simples, et il est à la base de nombreux algorithmes importants sur les graphes. L'algorithme de **Dijkstra** pour calculer les plus courts chemins à

origine unique et l'algorithme de **Prim** pour trouver l'arbre couvrant minimum utilisent des idées similaires à celles qui sont appliquées lors d'un parcours en largeur.

Étant donné un graphe $G = (V, E)$ et un sommet origine s , le parcours en largeur emprunte systématiquement les arcs de G pour découvrir tous les sommets accessibles depuis s . Il calcule la distance entre s et tous les autres sommets accessibles. Il construit également une arborescence de parcours en largeur de racine s , qui contient tous les sommets accessibles depuis s .

Pour tout sommet v accessible depuis s , le chemin reliant s à v dans l'arborescence de parcours en largeur correspond à un plus court chemin de s vers v dans G , autrement dit un chemin contenant le plus petit nombre d'arcs. L'algorithme fonctionne aussi bien sur les graphes orientés que sur les graphes non orientés.

L'algorithme de parcours en largeur tient son nom au fait qu'il découvre d'abord tous les sommets situés à une distance k de s avant de découvrir tout sommet situé à la distance $k + 1$.

Pour un meilleur suivi sur la progression du parcours, l'algorithme du parcours en largeur colorie chaque sommet en blanc, gris, ou noir. Au départ, tous les sommets sont considérés comme blancs, et peuvent devenir gris, puis noirs. Un sommet est découvert la première fois qu'il est rencontré au cours de la recherche et il perd alors sa couleur blanche pour devenir gris. Les sommets gris ou noirs ont donc été découverts, et la distinction entre gris et noir permet de garantir que la recherche se poursuit en largeur d'abord. Si $(u, v) \in E$ et si le sommet u est noir, alors le sommet v est gris ou noir ; autrement dit, tout sommet adjacent à un sommet noir a déjà été découvert. Les sommets gris peuvent être adjacents à des sommets blancs. Les sommets gris représentent donc la frontière entre les sommets découverts et les sommets non découverts.

Le parcours en largeur construit une arborescence de parcours en largeur, qui ne contient initialement que sa propre racine, représentant le sommet origine s . À chaque découverte d'un sommet v blanc pendant le balayage de la liste d'adjacences d'un sommet u déjà découvert, le sommet v et l'arc (u, v) sont ajoutés à l'arborescence. On dit alors que u est le prédécesseur ou parent de v dans l'arborescence de parcours en largeur. Comme un sommet est découvert au plus une fois, il possède au plus un parent. Les relations ancêtre/descendant dans l'arborescence de parcours en largeur sont définies relativement à la racine s de la manière habituelle : si u se trouve sur un chemin de l'arbre entre la racine s et le sommet v , alors u est un ancêtre de v et v est un descendant de u .

2.1 L'algorithme du parcours en largeur

Soit $G = (V, E)$ représenté par une liste d'adjacence et un sommet s choisi pour être la racine. Pour chaque sommet $u \in G$, on a les structures de données suivantes :

- *couleur* $[u]$ qui stocke sa couleur
- *parent* $[u]$ qui stocke l'identifiant de son parent; si u n'a pas de parent, alors *parent* $[u] = NIL$
- *d* $[u]$ qui est la distance calculée par l'algorithme entre le sommet u et le sommet origine s

- F une file pour gerer l'ensemble des sommets gris

Procédure **parcoursEnLargeur** (G, s)

1. pour chaque sommet $u \in G - s$, faire
 - $couleur[u] \leftarrow BLANC$
 - $d[u] \leftarrow \infty$
 - $parent[u] \leftarrow NIL$
2. $couleur[s] \leftarrow GRIS$
3. $d[s] \leftarrow 0$
4. $parent[s] \leftarrow NIL$
5. $F \leftarrow \{s\}$
6. Tant que $F \neq \emptyset$ Faire
 - $u \leftarrow tete(F)$
 - Pour chaque sommet $v \in Adj[u]$ faire
 - Si $couleur[v] \leftarrow BLANC$ alors
 - * $couleur[v] \leftarrow GRIS$
 - * $d[v] \leftarrow d[u] + 1$
 - * $parent[v] \leftarrow u$
 - * $Enfiler(F, v)$
 - $couleur[u] \leftarrow NOIR$
 - $Defiler(F)$

2.2 Analyse de l'algorithme

Théorème 1. *Le temps d'exécution de l'algorithme sur un graphe $G = (V, E)$ est $O(V + E)$.*

Bevis. Soit $G = (V, E)$ un graphe donné à notre algorithme en entrée. Après l'initialisation, aucun sommet n'est coloré en blanc et le test *Si* de la boucle garantit que chaque sommet est enfilé au plus une fois et, par temps défilé au plus une fois. Comme les opération d'enfilement et de défilement sont en $O(1)$, le temps total des opérations de la file est $O(V)$. Vu que la liste d'adjacence de chaque sommet n'est balayé que lorsque le sommet est défilé, alors, la liste d'ajacence de chaque sommet est parcourue au plus une fois. Comme la somme de longueurs de toutes les listes d'adjacence est $\theta(E)$, alors, le temps d'exécution total est $O(V + E)$. Le parcours en largeur s'exécute en un temps qui est linéaire par rapport à la taille de la représentation par liste d'adjacence de G .

Dans la suite de cette partie, il sera question de montrer que, comme dans un parcours en largeur, notre algorithme trouve la distance entre le sommet s et tout autre sommet du graphe. Soit $\delta(s, v)$ le nombre minimal d'arcs d'un chemin reliant s à v qui est aussi, la distance dans G entre s et v . S'il n'existe

pas de chemin entre s et t , on $\delta(s, v) = \infty$. On dit qu'un chemin de longueur $\delta(s, v)$ de s à t est un plus court chemin de s à t . Le lemme suivant sera utile pour la suite.

Lemme 1. *Soit $G = (V, E)$ un graphe orienté ou non, et soit $s \in V$ un sommet arbitraire. Alors, pour tout arc $(u, v) \in E$ on a : $\delta(s, v) \leq \delta(s, u) + 1$.*

Bevis. Si u est accessible depuis s , alors v l'est aussi. Dans ce cas, le plus court chemin de s à v ne peut pas être plus long que le plus court chemin de s à u , prolongé par l'arc (u, v) ; l'inégalité est donc valide. Si u ne peut pas être atteint depuis s , alors $\delta(s, v) = \infty$, et l'inégalité est vérifiée.

On cherche à montrer que pour n'importe quel sommet $v \in V$ $d[v] = \delta(s, v)$. Pour ce faire, nous allons d'abord montrer que $d[v]$ est un majorant de $\delta(s, v)$.

Lemme 2. *Soit $G = (V, E)$ un graphe tel que notre algorithme est exécuté sur G à partir d'un sommet s . Alors quand l'algorithme termine, on a pour tout sommet v de G , la valeur calculée $d[v]$ est telle que $d[v] \geq \delta(s, v)$.*

Bevis. A faire, ...

Pour démontrer que $d[v] = \delta(s, v)$, nous avons besoin de bien caractériser le comportement de la file F au cours de l'exécution. Nous avons besoin du lemme suivant:

Lemme 3. *Supposons que pendant l'exécution de l'algorithme sur un graphe $G = (V, E)$, la file F contient $\{v_1, v_2, \dots, v_r\}$, avec v_1 la tête de la file F et v_r la queue de la file F .*

Alors $d[v_r] \leq d[v_1] - 1$ et $d[v_i] \leq d[v_{i+1}] - 1$ pour $i = 1, 2, 3, \dots, r - 1$.

Bevis. A faire, ...

Comme les éléments de la file sont ajoutés successivement en partant de v_1 jusqu'à v_r . Alors, on a le corollaire suivant:

Corollaire 1. *Si deux sommets v_i et v_j sont enfilés et que v_i est enfilé avant v_j durant l'exécution. Alors $d[v_i] \leq d[v_j]$ au moment où v_j est enfilé.*

On est maintenant à même d'énoncer le théorème suivant:

Théorème 2. *Soit $G = (V, E)$ un graphe sur lequel, nous exécutons notre algorithme à partir d'un sommet s donné. Alors, pendant l'exécution, l'algorithme découvre chaque sommet v de V accessible à partir de s et à la fin on a $d[v] = \delta(s, v)$.*

2.3 Arborescence de parcours en largeur

Pendant le parcours du graphe, la procédure construit l'arborescence du parcours en largeur. Cet arborescence est représenté dans le champ *Parent*. Plus formellement, pour un graphe $G = (V, E)$ d'origine s on définit le sous graphe prédécesseur de G par $G_p = (V_p, E_p)$ où :

$V_p = \{v \in V : \text{parent}[v] \neq \text{NIL}\} \cup \{s\}$
et

$$E_p = \{(parent[v], v) \in E_p : v \in V_p - \{s\}\}$$

Le sous graphe prédécesseur $G_p = (V_p, E_p)$ est une arborescence de parcours en largeur si V_p est constitué des sommets accessibles depuis s . Tous chemin élémentaire de s à v dans G_p est un plus court chemin de s à v dans G .

En exécutant l'algorithme sur un graphe G et en supposant que le tableau *parent* est produit à la fin de cet exécution. L'algorithme ci-après permet d'imprimer les sommets d'un plus court chemin de s à v .

```

IMPRIMER_CHEMIN(G, s, v )
1  Si v = s alors
2    Imprimer (s)
3  Sinon si parent[v] = NIL alors
4    imprimer ("Il n'existe pas de chemin entre s et v")
5  Sinon IMPRIMER_CHEMIN(G, s, parent[v] )
6    imprimer (v)

```

Le temps d'exécution de cet algorithme est linéaire par rapport au nombre de sommets du chemin.

3 Parcours en profondeur

La stratégie suivie par un parcours en profondeur est, comme son nom l'indique, de descendre plus **profondément** dans le graphe chaque fois que c'est possible. Lors d'un parcours en profondeur, les arcs sont explorés à partir du sommet v découvert le plus récemment et dont on n'a pas encore exploré tous les arcs qui en partent. Lorsque tous les arcs de v ont été explorés, l'algorithme **revient en arrière** pour explorer les arcs qui partent du sommet à partir duquel v a été découvert. Ce processus se répète jusqu'à ce que tous les sommets accessibles à partir du sommet origine initial aient été découverts. S'il reste des sommets non découverts, on en choisit un qui servira de nouvelle origine, et le parcours reprend à partir de cette origine. Le processus complet est répété jusqu'à ce que tous les sommets aient été découverts. Comme dans le parcours en largeur, chaque fois qu'un sommet v est découvert pendant le balayage d'une liste d'adjacences d'un sommet u , le parcours en profondeur enregistre cet événement en donnant la valeur u à $p[v]$, parent de v . Contrairement au parcours en largeur, pour lequel le sous-graphe prédécesseur forme une arborescence, le sous-graphe prédécesseur obtenu par un parcours en profondeur peut être composé de plusieurs arborescences, car le parcours peut être répété à partir de plusieurs origines. Le sous-graphe prédécesseur d'un parcours en profondeur est donc défini un peu différemment de celui d'un parcours en largeur : on pose $G_p = (V, E_p)$, où

$$E_p = \{(parent[v], v) : v \in V_p \text{ et } parent[v] \neq NIL\}$$

Le sous-graphe prédécesseur d'un parcours en profondeur forme une forêt de parcours en profondeur composée de plusieurs arborescences de parcours en profondeur. Les arcs de E_p sont des arcs de liaison.

Comme dans le parcours en largeur, les sommets sont coloriés pendant le parcours pour indiquer leur état. Chaque sommet est initialement blanc, puis gris quand il est découvert pendant le parcours, et enfin noir en fin de traitement, c'est à dire quand sa liste d'adjacences a été complètement examinée.

Cette technique assure que chaque sommet appartient à une arborescence de parcours en profondeur et un seul, de sorte que ces arborescences sont disjointes.

En plus de créer une forêt de parcours en profondeur, le parcours en profondeur date chaque sommet.

Chaque sommet v porte deux dates : la première, $d[v]$, marque le moment où v a été découvert pour la première fois (et colorié en gris), et la seconde, $f[v]$, enregistre le moment où le parcours a fini d'examiner la liste d'adjacences de v (et le colorie en noir). Ces dates sont utilisées dans de nombreux algorithmes de graphes et sont utiles pour analyser le comportement du parcours en profondeur.

3.1 L'algorithme du parcours en profondeur

La procédure ci-après enregistre le moment où elle découvre le sommet u dans la variable $d[u]$, et le moment où elle termine le traitement du sommet u dans la variable $f[u]$. Ces dates sont des entiers compris entre 1 et $2|S|$, puisque découverte et fin de traitement se produisent une fois et une seule pour chacun des $|S|$ sommets. Pour tout sommet u ,

$$d[u] \leq f[u],$$

Le sommet u est BLANC avant l'instant $d[u]$, GRIS entre $d[u]$ et $f[u]$, et NOIR après. Le pseudo code suivant correspond à l'algorithme de base de parcours en profondeur. Le graphe d'entrée G peut être orienté ou non. La variable date est une variable globale qui sert à la datation.

PP(G)

```

1 Pour chaque sommet  $u$  de  $G$  Faire
2   couleur [ $u$ ] = BLANC
3   parent[ $u$ ] = NIL
4 date = 0
5 Pour chaque sommet  $u$  de  $G$  Faire
6   Si couleur [ $u$ ] == BLANC alors
7     visiter-PP( $u$ )
```

visiter-PP(u)

```

1   couleur [ $u$ ] = GRIS
2   date = date + 1
3    $d[u]$  = date
4   Pour chaque sommet  $v$  adjacent à  $u$  faire
5     Si couleur[ $v$ ] == BLANC alors
6       parent[ $v$ ] =  $u$ 
7       visiter-PP( $v$ )
8   couleur[ $u$ ] = NOIR
9    $f[u]$  = date
10  date = date + 1
```

Notez que le résultat du parcours en profondeur peut dépendre de l'ordre dans lequel les sommets sont examinés en ligne 5 de PP, et de l'ordre dans lequel les voisins d'un sommet sont visités en ligne 4 de VISITER-PP. Ces ordres de visite différents ne posent généralement pas problème dans la pratique, dans la

mesure où chaque résultat d'une recherche en profondeur fournit généralement des sorties équivalentes.

3.2 Propriétés du parcours en profondeur

Le parcours en profondeur révèle beaucoup d'informations sur la structure d'un graphe. La propriété la plus fondamentale du parcours en profondeur est sans doute que le sous-graphe prédécesseur G_p forme en fait une forêt, puisque la structure des arborescences de parcours en profondeur reflète exactement la structure des appels récurifs à VISITER-PP. Autrement dit, $u = p[v]$ si et seulement si VISITER-PP(v) a été appelé pendant le parcours de la liste d'adjacences de u . En outre, le sommet v est un descendant du sommet u dans la forêt de parcours en profondeur si et seulement si v est découvert pendant que u est gris.

Une autre propriété importante du parcours en profondeur tient au fait que les dates de découverte et de fin de traitement ont une structure parenthésée. Si l'on représente la découverte d'un sommet u par une parenthèse gauche « (u » et la fin de son traitement par une parenthèse droite « u) », alors la succession des découvertes et des fins de traitement crée une expression bien formée, au sens où les parenthèses sont correctement imbriquées. Une autre manière d'énoncer la condition de structure parenthésée est donnée par le théorème suivant.

Théorème 3. (*Théorème des parenthèses*) Dans un parcours en profondeur d'un graphe $G = (V, E)$ (orienté ou non), pour deux sommets quelconques u et v , une et une seule des trois conditions suivantes est vérifiée :

- les intervalles $[d[u], f[u]]$ et $[d[v], f[v]]$ sont complètement disjoints, et ni u ni v n'est un descendant de l'autre dans la forêt de parcours en profondeur,
- l'intervalle $[d[u], f[u]]$ est entièrement inclus dans l'intervalle $[d[v], f[v]]$, et u est un descendant de v dans une arborescence de parcours en profondeur, ou
- l'intervalle $[d[v], f[v]]$ est entièrement inclus dans l'intervalle $[d[u], f[u]]$, et v est un descendant de u dans une arborescence de parcours en profondeur.

Corollaire 2. (*Imbrications des intervalles des descendants*) Le sommet v est un descendant propre du sommet u dans la forêt de parcours en profondeur d'un graphe G (orienté ou non) si et seulement si $d[u] < d[v] < f[v] < f[u]$.

Le théorème suivant donne une autre caractérisation importante des descendants d'un sommet dans une forêt de parcours en profondeur.

Théorème 4. (*Théorème du chemin blanc*) Dans une forêt de parcours en profondeur d'un graphe $G = (V, E)$ (orienté ou non), un sommet v est un descendant d'un sommet u si et seulement si, au moment $d[u]$ où le parcours découvre u , il existe un chemin de u à v composé uniquement de sommets blancs.

3.3 Classification des arcs

Une autre propriété intéressante du parcours en profondeur est que le parcours peut servir à classer les arcs du graphe d'entrée $G = (V, E)$. Cette classification des arcs peut servir à glaner d'importantes informations concernant le graphe. Par exemple, dans la section suivante, nous verrons qu'un graphe orienté est acyclique si et seulement si un parcours en profondeur n'engendre aucun arc « arrière ».

Il est possible de définir quatre types d'arcs en fonction de la forêt de parcours en profondeur G_p obtenue par un parcours en profondeur sur G .

- Les arcs de liaison sont les arcs de la forêt de parcours en profondeur G_p . L'arc (u, v) est un arc de liaison si v a été découvert la première fois pendant le parcours de l'arc (u, v) .
- Les arcs arrières sont les arcs (u, v) reliant un sommet u à un ancêtre v dans une arborescence de parcours en profondeur. Les boucles (graphes orientés uniquement) sont considérées comme des arcs arrières.
- Les arcs avants sont les arcs (u, v) qui ne sont pas des arcs de liaison, et qui relient un sommet u à un descendant v dans une arborescence de parcours en profondeur.
- Les arcs transverses sont tous les autres arcs. Ils peuvent relier deux sommets d'un même arbre de parcours en profondeur, du moment que l'un des sommets n'est pas un ancêtre de l'autre ; ils peuvent aussi relier deux sommets appartenant à des arborescences de parcours en profondeur différentes.